Documentation — Personalized Website Chatbot with RAG and Database

📌 Overview

This project implements a **Retrieval-Augmented Generation (RAG)-based chatbot** for a website that uses a **database and vector search** to deliver personalized, context-aware responses.
It's built in **Python** using a **Streamlit UI** and backend logic to handle embeddings, retrieval, and LLM answering—all designed to make an intelligent chatbot that understands and retrieves information from stored content.

A RAG chatbot enhances traditional LLM responses by retrieving relevant chunks of information from a searchable knowledge source (like a database) before generating an answer.

▯ Key Features

- **Website chatbot with RAG support** — Answers based on relevant context instead of generic replies.
- **Vector database / DB-backed search** — Uses a database or vector index to find semantically relevant content.
- **Streamlit Web App** — Interactive chat interface for users to type questions and receive answers.
- **Database storage** — Stores content, conversations, or embeddings for persistent retrieval.

⚙️ Architecture (High-Level)

1. **Content Ingestion**
    - Text or documents are processed and split into chunks.
    - Text chunks are converted into embeddings (numeric vectors).
2. **Vector Database**
    - Embeddings are stored in a vector store (e.g., FAISS, Chroma, or similar).
    - At query time, the chatbot searches for similar embeddings to find relevant content.
3. **Query & Generation**
    - The user's question goes through a retriever-reader pipeline.
    - The retriever selects relevant content and passes it to the LLM.
    - The LLM generates an answer conditioned on the retrieved context.

🚀 Quick Start

1. Clone the Repo

```
git clone https://github.com/ANAS-alt-ctr/-Personalized-Website-Chatbot-with-RAG-and-Database
cd -Personalized-Website-Chatbot-with-RAG-and-Database
```

---

2. Install Dependencies

Assuming Python and pip are installed:

```
pip install -r requirements.txt
```

Install any additional database or vector store dependencies your setup requires.

---

3. Configure Environment

- Create a .env file or configure environment variables for:
  - Gemini API keys (if using Gemini for LLM)
  - Database credentials
  - Vector database config

---

4. Run the App

```
streamlit run streamlit_app.py
```

🛠 How It Works (Simplified)

1. **User Message**
   User enters a query in the web interface.
2. **Retrieval**
   The system embeds the query and searches the vector store for relevant vectors (text chunks).
3. **Generation**
   The chatbot uses a language model to generate a response using the retrieved context.

□ Project Structure

```
├── app/                # Application logic (backend + database code)
├── streamlit_app.py    # Main Streamlit UI entrypoint
├── .gitignore
├── requirements.txt    # Python dependencies
```

(Exact structure may vary; adjust based on repository contents.)

📌 Notes

- The repository currently has **no official README description** — so most explanation comes from inspecting the code files.
- The RAG concept used here is similar to many open RAG chatbot examples where embeddings and semantic search improve response relevance.

📚 Learn More

If you need deeper background on RAG chatbots:

- Retrieval-Augmented Generation (RAG) combines vector search with LLM generation

ERRORS:

In the embedding I solve it with sentence Transformer.

In LLM I solve it with change the model 1.5 flash into 2.5 flash

In scraping I solve it with reinstall the bs4